

Code Layout, Readability, and Reusability

DATE	02 JULY 2026
TEAM ID	
PROJECT NAME	A Comprehensive Measure of Well-Being: Human Development Index (HDI) Predictor
MAXIMUM MARKS	5 MARKS

1. Overview

This document specifies the software architecture guidelines, code layout, readability features, and reusability models applied in the development of the Human Development Index (HDI) Prediction System. These coding standards ensure a clean, maintainable, and readable codebase.

2. Directory and Code Layout

The project structure keeps logic, presentation, data, and models separate, isolating concerns across folders:

- **Root Directory** (``5.Project Development Phase/code/``): Contains Python script entry points (``app.py``, ``ml_model.py``, ``data_preprocessing.py``).
- **Data Folder** (``data/``): Stores input CSV datasets.
- **Models Folder** (``models/``): Stores pickled serialized models (``hdi_model.pkl``) and label encoders (``encoder.pkl``).
- **Templates Folder** (``templates/``): Holds Jinja2 HTML layout pages.
- **Static Assets** (``static/``): Holds CSS stylesheets and visual assets.

2.1 Code Flow Design

- Processing starts in the data layer where features are encoded.
- The model trains and outputs pickled metrics.
- The application server loads these models, intercepts POST queries, transforms parameters, and returns visual HTML dashboards to the browser.

3. Readability & Coding Style

To ensure readability for new developers, the code follows these standards:

3.1 PEP 8 Standards Compliance

- Uses standard 4-space indentation.
- Standard imports are organized at the top of each file, grouped by standard library, third-party libraries, and local modules.
- Clean spacing is used around operators, inside expressions, and between class and function blocks.

3.2 Clear Naming Conventions

- Variables use lowercase snake_case (e.g., `life\_expectancy`, `expected\_years\_schooling`).
- Functions use descriptive verbs (e.g., `classify\_hdi`, `get\_hdi\_insights`, `load\_data`).
- Constants use UPPERCASE snake_case (e.g., `BASE\_DIR`, `MODEL\_PATH`).

3.3 Comments and Documentation

- Every function includes a detailed docstring explaining inputs, outputs, exceptions, and overall logic.
- Complex blocks (such as model path resolutions or boundary checks) include step-by-step inline comments.

4. Reusability Model

To minimize duplicate code, the application uses modular functions:

- **Data Processing Functions:** The preprocessing steps in `data\_preprocessing.py` are used during both model training (`ml\_model.py`) and live inference (`app.py`).
- **Design Token System:** All color values, font families, and visual variables are defined as CSS variables at the root of `style.css`:

```
``css

:root {

  --bg-gradient: linear-gradient(135deg, #0f172a 0%, #1e1b4b 100%);

  --card-bg: rgba(30, 41, 59, 0.7);

  --text-main: #f8fafc;

  --accent-blue: #2563eb;

}
```

...

- **Configuration-Driven Insights:** Policy recommendations are stored in a key-value dictionary structure within `app.py`, making it easy to edit or add recommendations in the future without changing the code logic.